Understanding Static Methods and Data

Introduction

Static keyword is widely used to ***share the same field or method*** among all the objects of the class. The actual goal of the `static` keyword is to share a single data over all the objects.

There are three types of sharing using the **`static`** keyword. They are:

- `Static Method`
- `Static Field`
- `Static Class`

Now I am going to give a short brief on these three types.

Static Method

A `static` method can be accessed from outside the class without creating any object of this class. This `static`method can be accessed directly by the name of the `static` method followed by the . (dot operator) and the class name.

For example, we can consider the `Sqrt` method of the `Math` class. This is how the `Sqrt` method of the real `Math` class is defined :

Hide Copy Code

```
class Math
{
// Do something
    public static double Sqrt(double d)
    {
      // Do something
    }
}
```

From the previous example, you can notify that the `Sqrt` method is declared as `static` so that it can be accessed by using the class name directly, no object of the `Math` class is required to access the `static` method.

So, here I will like to some general properties of a `static` method:

- It can access only the `static` fields of the class.
- It can directly invoke the methods that are defined as `static`.

Static Fields

A `static` field is shared among all the objects of the of the class. So if an object changes this value, then all the objects of this class will get the changed value.

Look at the example below:

Hide Copy Code

```
class Point
    {
        public Point()
        {
            this.x = -1;
            this.y = -1;
            Console.WriteLine("Deafult Constructor Called");
            objectCount++;
        }

        public Point(int x, int y)
        {
            this.x = x;
            this.y = y;
            Console.WriteLine("x = {0} , y = {1}", x, y);
            objectCount++;
        }

       private int x, y;
       public static int objectCount = 0;
   }
```

Now if we create three objects for this class:

Hide Copy Code

```
Point origin1 = new Point();         // objectCount = 1
Point origin2 = new Point();         // objectCount = 2
Point origin3 = new Point();         // objectCount = 3
```

Here these three objects share the same field so every time the `static` field `objectCount` is incremented.

Static Class

A `static` class is used to hold all the utility methods and fields. A `static` Class has some properties:

- All the methods and fields inside the class must be declared as `static`.
- A `static` class cannot contain any instance method or data.
- No object can be created (even using the "`new`" keyword) of this class.
- It can have a default constructor and it is also `static`.

Example

Let us consider an example.

Point.cs

Hide Shrink Copy Code

```
using System;
using System.Collections.Generic;
using System.Text;

namespace ConAppCH7CCons
{
    class Point
    {
        public Point()
        {
            this.x = -1;
            this.y = -1;
            Console.WriteLine("Deafult Constructor Called");
            objectCount++;
        }
        public Point(int x, int y)
        {
            this.x = x;
            this.y = y;
            Console.WriteLine("x = {0} , y = {1}", x, y);
            objectCount++;
        }
        public double DistanceTo(Point other)
        {
            int xDiff = this.x - other.x;
            int yDiff = this.y - other.y;
            return Math.Sqrt((xDiff * xDiff) + (yDiff * yDiff));
        }

        public static int FnObjectCount()
        {
            return objectCount;
        }

        private int x, y;
        private static int objectCount = 0;
    }
}
```

ConstructorExample.cs

Hide Copy Code

```
using System;
using System.Collections.Generic;
using System.Text;

namespace ConAppCH7CCons
```

```
{
    class ConstructorExample
    {
        static void Main(string[] args)
        {
            Point origin = new Point();
            Point bottomRight = new Point(1024, 1280);
            double distance = origin.DistanceTo(bottomRight);
            Console.WriteLine("distance = {0}",distance);
            Console.WriteLine("No of Objects {0}", Point.FnObjectCount());
            Console.ReadLine();
        }
    }
}
```

In this example, the `FnObjectCount()` method is called directly with the class name, no object is required here.

If you want to access the `static` method with an object like:

Hide Copy Code

```
bottomRight.FnObjectCount()          // do not write this
```

then the compiler will report an error.

Now, take a deeper look into the `FnObjectCount()` `static` method:

Hide Copy Code

```
public static int FnObjectCount()
{
  return objectCount;
}
```

As it is a `static` method, it can hold only a `static` field or `static` method.